

Constraint Satisfaction Problems

Constraint satisfaction problems (CSPs) are problems in which the search is for a **feasible** solution, meaning one that satisfies all the constraints. By definition, constraint satisfaction involves *hard* constraints, meaning constraints that are either satisfied or not, with no middle ground. Nonetheless, constraint satisfaction problems are often solved as optimization problems with *soft* constraints. The objective is to maximize the number of constraints satisfied; or equivalently, to minimize the number of constraints violated.

Examples of CSPs include crossword puzzles and cryptarithmic puzzles like these:

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array} \qquad \begin{array}{r} \text{T W O} \\ + \text{T W O} \\ \hline \text{F O U R} \end{array}$$

In these examples, the M and the F are both constrained to be 1, since the convention is that leading 0s are dropped. This constraint then propagates: in the first example, $S + 1 = 10$; likewise, in the second example, $T + T = 10$. The values of S and T are thus constrained, and constraint propagation can continue.

1 CSPs

Formally, a (finite) **constraint satisfaction problem** is a triple $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$, where

- a finite set of variables $\mathcal{X} = \{x_1, \dots, x_n\}$
- a set of finite domains $\mathcal{D} = \{D_1, \dots, D_n\}$
- a finite set of constraints, meaning a set of variables along with permissible combinations of their values

A **solution** to a CSP is a complete and consistent assignment, where

- An **assignment** is a mapping from variables to values.
- A **complete** assignment assigns a value to all variables.
- A **consistent** assignment does not violate any constraints.

Constraints can be expressed *intensionally* or *extensionally*. Given a CSP with two variables x_1 and x_2 , if $D(x_1) = \{A, B\}$ and $D(x_2) = \{B, C\}$, then the constraint “the value of x_1 cannot equal the value of x_2 ” is expressed intensionally as $x_1 \neq x_2$, and extensionally as $\{\{x_1 \mapsto A, x_2 \mapsto B\}, \{x_1 \mapsto A, x_2 \mapsto C\}, \{x_1 \mapsto B, x_2 \mapsto C\}\}$. The latter representation enumerates all consistent assignments.

Example SAT is a canonical example of a CSP. Consider a Boolean formula expressed in CNF: e.g., $A \vee \neg B \vee C$. The domain of the variables A , B , and C is $\{T, F\}$, while $A \vee \neg B \vee C$ defines the constraint intensionally. The extensional representation comprises all assignments except $\{A \mapsto F, B \mapsto T, C \mapsto F\}$.

Example Another classic example of a CSP is the n -queens problem. In this problem, n queens are to be placed on an $n \times n$ chess board, each on her own square, *s.t.* no two queens threaten each other: i.e., no two occupy the same row, column, or diagonal. Solutions to sample n -queens problems are depicted in Figure 1.

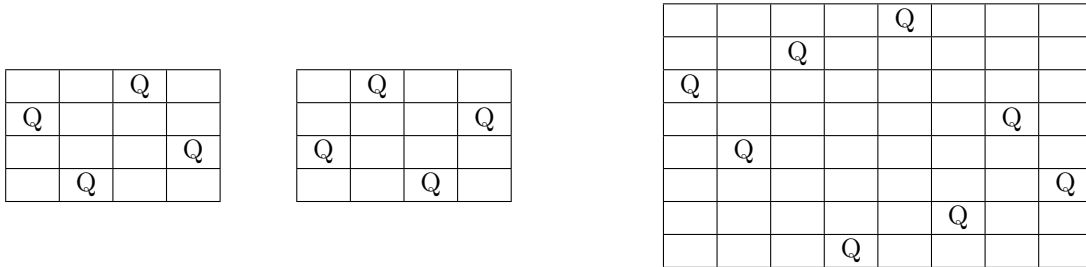


Figure 1: Solutions to the n -queens problem for $n = 4$ and $n = 8$.

Given an $n \times n$ chess board, the n -queens problem can be represented as a CSP as follows:

- Variables: r_i and c_i , for $i \in \{1, \dots, n\}$, where r_i (resp. c_i) represents the i th queen's row (resp. column)
- Domains: The domain of all variables is $\{1, \dots, n\}$.
- Constraints:

- No two queens can occupy the same row: i.e., for $i \neq j \in \{1, \dots, n\}$, $r_i \neq r_j$.
- Likewise, no two queens can occupy the same column: i.e., for $i \neq j \in \{1, \dots, n\}$, $c_i \neq c_j$.

Note that the aforementioned constraints imply that no two queens can occupy the same square.

- No two queens can occupy the same diagonal or antidiagonal. (The diagonals run from the top left of the board to the bottom right & the anti-diagonals, from the top right to the bottom left.)

The difference between the indices in a matrix is constant along the diagonals. Similarly, the sum of the indices of a matrix is constant along the anti-diagonals. For example, in a 4×4 matrix,

	1	2	3	4		1	2	3	4
1	0	-1	-2	-3	1	2	3	4	5
2	1	0	-1	-2	2	3	4	5	6
3	2	1	0	-1	3	4	5	6	7
4	3	2	1	0	4	5	6	7	8

If queen i resides at $(1, 3)$ and queen j resides at $(2, 4)$, then they occupy the same diagonal. More generally, two queens $i \doteq (r_i, c_i)$ and $j \doteq (r_j, c_j)$ occupy the same diagonal iff $r_i - c_i = r_j - c_j$.

If queen i resides at $(3, 2)$ and queen j resides at $(4, 1)$, then they occupy the same *anti*-diagonal. Two queens $i \doteq (r_i, c_i)$ and $j \doteq (r_j, c_j)$ occupy the same *anti*-diagonal iff $r_i + c_i = r_j + c_j$.

The requisite constraints are thus, for $i \neq j \in \{1, \dots, n\}$, $r_i - c_i \neq r_j - c_j$ and $r_i + c_i \neq r_j + c_j$.

The assignment $\{r_1 \mapsto 2, c_1 \mapsto 1, r_2 \mapsto 4, c_2 \mapsto 2, r_3 \mapsto 1, c_3 \mapsto 3, r_4 \mapsto 3, c_4 \mapsto 4\}$ solves the 4-queens problem, but this assignment is not unique.

One approach to solving CSPs is via **constraint programming**, which extends blind search (i.e., backtracking) methods with **constraint propagation** heuristics suited to CSPs. These heuristics proactively eliminate extensions of assignments that are inconsistent with partial assignments made earlier in the search. For example, if A is assigned the value F, but one of A or B must be T, then a constraint propagation algorithm would eliminate from consideration all extensions that assign B the value F.

We will pursue a different solution approach: we will reduce an instance of a CSP to an instance of SAT, and then run a SAT solver in search of a solution to the CSP. In the early 2000s, when these notes were originally compiled, WALKSAT (i.e., local search) was a more or less state-of-the-art SAT solver—especially on randomly-generated SAT instances. Today’s best SAT solvers instead employ systematic search and constraint propagation—enhanced with variable ordering heuristics, some of which use machine learning to decide the order in which to assign variables to values.

This is what it means to reduce one problem to another:

1. Given an instance x of a problem A
2. Apply the reduction f to x , which yields $f(x)$, an instance of SAT
3. Run a SAT solver on $f(x)$, which outputs z
4. Decompile z into a solution to A

In this lecture, we reduce n -queens and graph coloring to SAT.

2 Aside: Cardinality Constraints

To carry out the reductions, we need some practice using propositional logic to express constraints. Given a set of variables, say $\{x_1, x_2, \dots, x_n\}$, let’s express the following:

1. At least one variable is true.
2. At most one variable is true.
3. Exactly one variable is true.

At least one variable is true This constraint is straightforward to express:

$$x_1 \vee x_2 \vee x_3 \vee \dots \vee x_n$$

Note that this constraint is in CNF: it comprises a single conjunct, which is itself a disjunction.

At most one variable is true Equivalently, no two variables, such as x_1 and x_2 , can both be true: $\neg(x_1 \wedge x_2)$. By De Morgan’s law, this formula can be equivalently expressed in CNF as $\neg x_1 \vee \neg x_2$.

More generally,

$$\neg(x_1 \wedge x_2) \wedge \neg(x_1 \wedge x_3) \wedge \dots \wedge \neg(x_{n-1} \wedge x_n)$$

Equivalently, in CNF,

$$(\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge \dots \wedge (\neg x_{n-1} \vee \neg x_n)$$

Exactly one variable is true At least one variable is true and at most one variable is true:

$$(x_1 \vee x_2 \vee \dots \vee x_n) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge \dots \wedge (\neg x_{n-1} \vee \neg x_n)$$

As this constraint is the conjunction of the two previous ones, both in CNF, this one is likewise in CNF, i.e., as a conjunction of disjunctions of literals.

3 Graph Coloring

Perhaps you've heard of the famous four-color theorem? The theorem concerns maps and their colorings. Given a map, a **coloring** is an assignment of colors to regions on the map such that no two adjacent regions are the same color. The four-color theorem states that no more than four colors are required to color any map! This theorem is famous because it was the first theorem whose proof was established with help from a computer—in 1976! [1, 2]

The map-coloring problem can be reformulated as **graph coloring**, the problem of coloring a graph so that no adjacent vertices are the same color. In this reformulation, every region on the map is represented as a vertex in the graph, and every (non-negligible) border is represented by an edge connecting the regions/vertices that share that border. A coloring of the map then corresponds to a coloring of the graph. See Figure 2 for an example of a colored map and its corresponding graph.

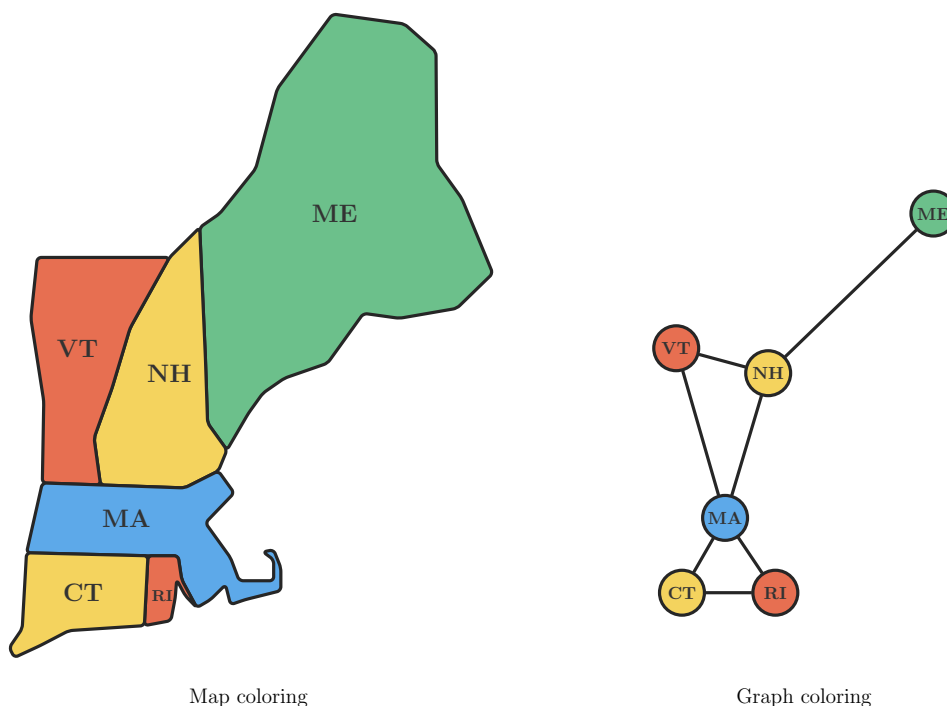


Figure 2: A colored map of New England and its corresponding colored graph.

Many real-world problems can be represented as graph coloring, such as final exam scheduling. Imagine a university that seeks to schedule exams for its class offerings, such that no student has a conflict. We can create a graph whose nodes are the classes with edges between two nodes/classes whenever there is a student taking both classes. We can now attempt to color this graph with colors representing exam slots. If we are successful, since no adjacent nodes are the same color, no student has a scheduling conflict.

Definition A graph is said to be **properly colored** if no two adjacent vertices are the same color.

Let's describe graph coloring formally as a CSP, and then reduce it to SAT.

Given a graph $G = (V, E)$ with vertices V and edges E , graph coloring as a CSP comprises:

- Variables: For all $v \in V$, a variable x_v .
- Domains: The variables' domain is the set of colors. Assuming k colors, the domain $D = \{1, \dots, k\}$.
- Constraints: For all edges $e = (u, v) \in E$, $x_u \neq x_v$.

Next, we reduce graph coloring to SAT, by expressing the problem as a Boolean formula in CNF.

To start, we choose a set of variables. Recall that the meaning (i.e., domain value) of a variable in graph coloring is its color. In SAT, we are restricted to propositional variables, whose domain is $\{T, F\}$, but we aim to express this same meaning. We can accomplish this by blowing up the number of variables, from $|V|$ to $k|V|$, so that each variable x_{vc} indicates whether vertex v is colored c .

Having defined our variables, we now move on to formulating the constraints.

- **Each vertex is assigned exactly one color**

For each vertex $v \in V$, $(x_{v1} \vee x_{v2} \vee \dots \vee x_{vk}) \wedge (\neg x_{v1} \vee \neg x_{v2}) \wedge (\neg x_{v1} \vee \neg x_{v3}) \wedge \dots \wedge (\neg x_{v,k-1} \vee \neg x_{v,k})$.

- **Adjacent vertices cannot be assigned the same color**

For all edges $(u, v) \in E$ and all colors $c \in D$, $\neg(x_{uc} \wedge x_{vc})$. Or, equivalently, $\neg x_{uc} \vee \neg x_{vc}$.

As all our constraints are disjunctions, their conjunction is our desired representation of graph coloring as a CNF formula. Moreover, the given graph G can be properly colored iff the SAT encoding of G is satisfiable. (Proof of this claim is left as an exercise to the reader.) We have therefore reduced graph coloring to SAT.

4 Solving n -Queens via SAT

In our earlier formulation of n -queens, the queens' locations were the variables, and the domain of these variables was $\{1, \dots, n\}$. SAT variables, however, are propositions, whose domains are necessarily $\{T, F\}$. Therefore, we must change our representation (i.e., the way we interpret the variables in our formulation) from a pair of indices that indicate a location to something that preserves this meaning (i.e., the location), but evaluates to T or F. We can do this by blowing up our representation from $2n$ variables to n^2 variables, one per grid cell. For all rows, columns $i, j \in \{1, \dots, n\}$, we define x_{ij} such that x_{ij} is T if a queen occupies the i th row and the j th column. Otherwise, x_{ij} is F.

Having defined our variables, we now move on to formulating the constraints. Our goal is to construct a formula in CNF. As long as we can construct each individual constraint as a disjunction of literals (a clause), we can enforce them all by considering their conjunction, which will be a formula in CNF.

There must be exactly one queen in each row. There must be at least one queen in each row, and there can be at most one queen in each row.

- **At least one queen in each row**

For all rows $1 \leq i \leq n$, $x_{i,1} \vee x_{i,2} \vee \dots \vee x_{i,n}$.

How many such constraints are there? There are n of these constraints, since there are n rows.

- **At most one queen in each row**

For all rows $1 \leq i \leq n$ and all pairs of distinct columns $1 \leq j < k \leq n$, $\neg x_{ij} \vee \neg x_{ik}$.

How many such constraints are there? There are $O(n^3)$ such constraints, since there are n rows and there are $n \times n - 1$ pairs of distinct columns in each row.

There must be exactly one queen in each column. This constraint is analogous to the previous constraint, but we represent it by iterating over columns instead of rows.

- **At least one queen in each column**

For all columns $1 \leq j \leq n$, $x_{1j} \vee x_{2j} \vee \dots \vee x_{nj}$.

- **At most one queen in each column**

For all columns $1 \leq j \leq n$ and all pairs of distinct rows $1 \leq i < k \leq n$, $\neg x_{ij} \vee \neg x_{kj}$.

There cannot be two queens on the same diagonal or anti-diagonal.

Diagonals For all pairs (i, j) and (k, l) s.t. $i - j = k - l$, $i \neq k$, and $j \neq l$, $\neg x_{ij} \vee \neg x_{kl}$.

Anti-diagonals For all pairs (i, j) and (k, l) s.t. $i + j = k + l$, $i \neq k$, and $j \neq l$, $\neg x_{ij} \vee \neg x_{kl}$.

How many such constraints are there? Each constraint involves two variables, so the question is, how many pairs of variables are there? Naively, we might imagine there are n^2 choices for the (i, j) pairs and another n^2 choices for the (k, l) pairs. But in fact, the choices are restricted by the constraints. e.g., $l = j - i + k$ —there are only actually three degrees of freedom—so we need not consider any more than $O(n^3)$ possibilities.¹

We have represented all of the n -queen constraints as disjunctions. Their conjunction is thus a representation in CNF of the n -queens problem. If we can find a satisfying assignment for this CNF formula, then we will have found a solution to the n -queens problem! Likewise, if we *cannot* find a satisfying assignment for this CNF formula—which we cannot for $n = 2$ or $n = 3$ —then a solution to the n -queens problem does not exist!

References

- [1] Kenneth Appel and Wolfgang Haken. Every planar map is four colorable, i: Discharging. *Illinois Journal of Mathematics*, 21(3):429–490, 1977.
- [2] Kenneth Appel, Wolfgang Haken, and John Koch. Every planar map is four colorable, ii: Reducibility. *Illinois Journal of Mathematics*, 21(3):491–567, 1977.

¹Naively enumerating all n^3 possibilities is also overkill, since $\neg x_{12} \vee \neg x_{34}$ is equivalent to $\neg x_{34} \vee \neg x_{12}$, but even after accounting for such symmetries, the total number of constraints remains $O(n^3)$.